

GESTION DES NOTES

Académie d'Éducation

Rapport de Projet

Réalisé par :

EL MAZYANI MOHAMED ILYAS
HAMZA ABAGAR
BRIK AOUSSAR

Encadré par :

Dr. Khammal

Version : 1.0

Date : 17 juin 2026

Table des matières

1	Introduction	3
1.1	Présentation du projet	3
1.2	Objectifs	3
1.3	Acteurs du système	3
2	Architecture du projet	4
2.1	Architecture générale	4
2.2	Technologies utilisées	4
3	Diagramme de classe	5
3.1	Description des classes	5
3.2	Structure des classes	5
3.2.1	Classe Utilisateur	5
3.2.2	Classe Etudiant	6
3.2.3	Classe Enseignant	7
3.2.4	Classe Module	7
3.2.5	Classe SousModule	8
3.2.6	Classe Note	9
3.2.7	Classe Promotion	10
4	Structure de la base de données	12
4.1	Schéma de la base de données	12
4.2	Requêtes principales	12
4.2.1	Authentification	12
4.2.2	Liste des étudiants	12
5	Règles de validation	13
5.1	Règlement d'évaluation	13
5.2	Calcul des moyennes	13
5.3	Algorithme de validation	14
6	Interface utilisateur	16
6.1	Page de connexion	16
6.2	Interface principale	16
6.3	Gestion des étudiants	16
6.4	Statistiques	17
6.5	Structure du menu	17
7	Import/Export Excel	18
7.1	Fonctionnalités	18
7.2	Structure du fichier Excel	18
7.3	Code d'export	18
8	Détails de la base de données	19
8.1	Table Utilisateur	19
8.2	Table Étudiant	19

9 Conclusion	20
9.1 Bilan du projet	20
9.2 Points forts	20
10 Annexes	21
10.1 Configuration du projet	21
10.2 Dépendances principales	21

1 Introduction

1.1 Présentation du projet

Ce projet consiste en une application de gestion des notes destinée aux établissements d'enseignement supérieur. Développée par EL MAZYANI MOHAMED ILYAS, HAMZA ABAGAR et BRIK AOUSSAR, l'application permet aux différents acteurs (étudiants, enseignants, responsables de filière et personnel du planning) de gérer efficacement les notes, les promotions, les modules et les évaluations.

1.2 Objectifs

- Centraliser la gestion des notes des étudiants
- Automatiser le calcul des moyennes et des validations
- Offrir une interface intuitive pour chaque type d'utilisateur
- Assurer le suivi de la progression académique des étudiants

1.3 Acteurs du système

- **PLANNING** : Gestion complète des étudiants, enseignants, promotions, modules et notes
- **RESPONSABLE_FILIERE** : Gestion des étudiants, validation des années et consultation des statistiques
- **ENSEIGNANT** : Saisie et gestion des notes
- **ETUDIANT** : Consultation des notes et suivi de la progression

2 Architecture du projet

2.1 Architecture générale

Le projet suit une architecture en couches avec une interface utilisateur JavaFX, une couche métier et une couche d'accès aux données MySQL.

Interface Utilisateur (JavaFX)
Couche Métier (Java)
Couche Données (MySQL)

FIGURE 1 – Architecture en couches

2.2 Technologies utilisées

- **Java** : Langage de programmation principal
- **JavaFX** : Framework pour l'interface utilisateur
- **MySQL** : Base de données relationnelle
- **Apache POI** : Import/Export des fichiers Excel

3 Diagramme de classe

3.1 Description des classes

TABLE 1 – Classes principales du projet

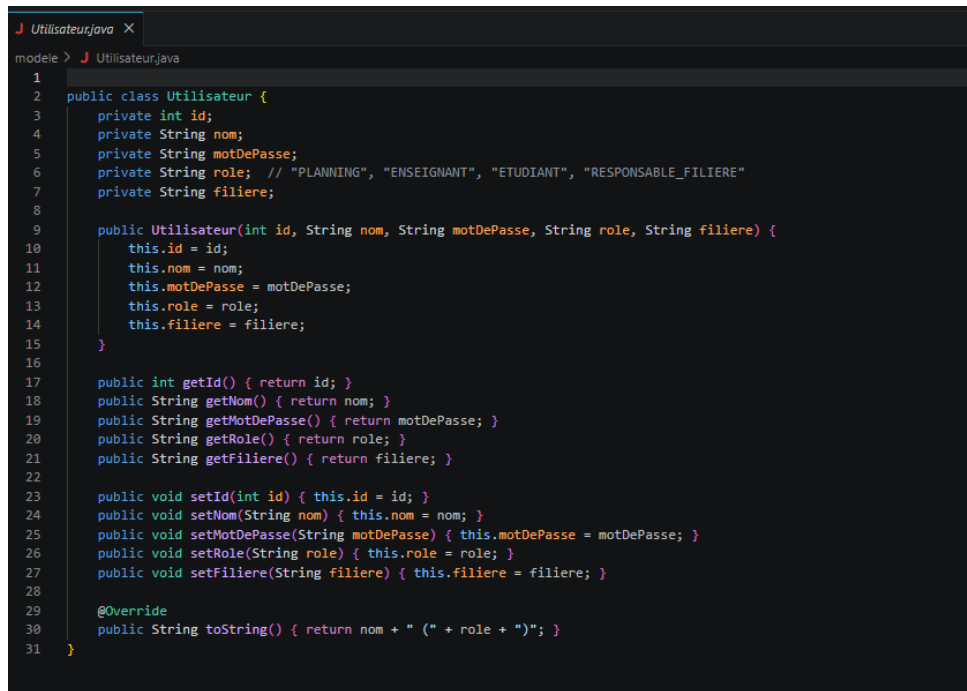
Classe	Rôle
Utilisateur	Gestion des utilisateurs et authentification
Etudiant	Représentation d'un étudiant
Enseignant	Représentation d'un enseignant
Promotion	Gestion des promotions
Module	Gestion des modules d'enseignement
SousModule	Gestion des sous-modules
Note	Gestion des notes
StockageMySQL	Accès à la base de données
InterfaceFx	Interface utilisateur JavaFX
ReglementEvaluation	Règles de validation

3.2 Structure des classes

3.2.1 Classe Utilisateur

```
1 public class Utilisateur {  
2     private int id;  
3     private String nom;  
4     private String motDePasse;  
5     private String role;  
6     private String filiere;  
7     // Getters et Setters  
8 }
```

Listing 1 – Classe Utilisateur

A screenshot of a Java IDE window titled 'Utilisateur.java'. The code defines a 'Utilisateur' class with private attributes 'id', 'nom', 'motDePasse', 'role', and 'filiere'. It includes a constructor, getter and setter methods for each attribute, and an overridden 'toString()' method. The code is as follows:

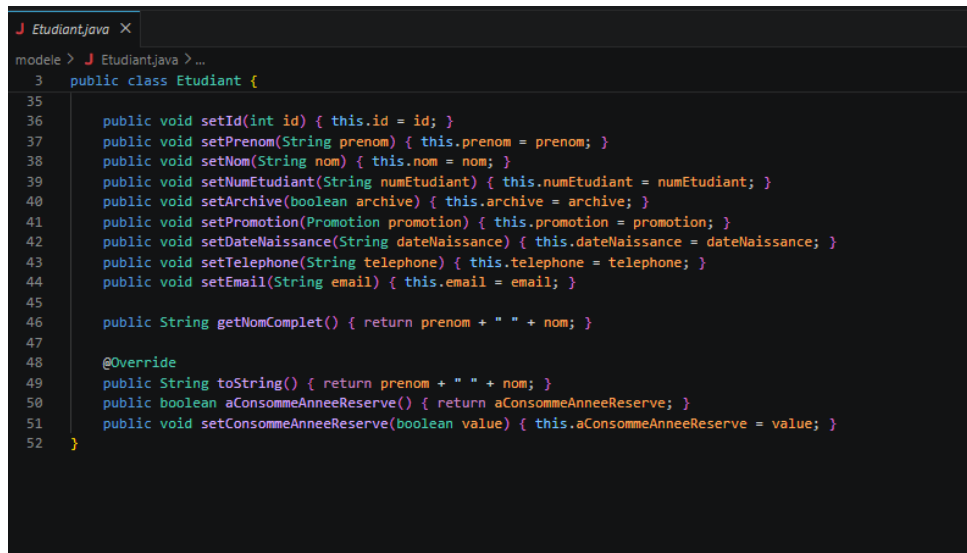
```
1 public class Utilisateur {
2     private int id;
3     private String nom;
4     private String motDePasse;
5     private String role; // "PLANNING", "ENSEIGNANT", "ETUDIANT", "RESPONSABLE_FILIERE"
6     private String filiere;
7
8     public Utilisateur(int id, String nom, String motDePasse, String role, String filiere) {
9         this.id = id;
10        this.nom = nom;
11        this.motDePasse = motDePasse;
12        this.role = role;
13        this.filiere = filiere;
14    }
15
16    public int getId() { return id; }
17    public String getNom() { return nom; }
18    public String getMotDePasse() { return motDePasse; }
19    public String getRole() { return role; }
20    public String getFiliere() { return filiere; }
21
22    public void setId(int id) { this.id = id; }
23    public void setNom(String nom) { this.nom = nom; }
24    public void setMotDePasse(String motDePasse) { this.motDePasse = motDePasse; }
25    public void setRole(String role) { this.role = role; }
26    public void setFiliere(String filiere) { this.filiere = filiere; }
27
28    @Override
29    public String toString() { return nom + " (" + role + ")"; }
30 }
31
```

FIGURE 2 – Classe Utilisateur

3.2.2 Classe Etudiant

```
1 public class Etudiant {
2     private int id;
3     private String prenom;
4     private String nom;
5     private String numEtudiant;
6     private boolean archive;
7     private Promotion promotion;
8     private String dateNaissance;
9     private String telephone;
10    private String email;
11    // Getters et Setters
12 }
```

Listing 2 – Classe Etudiant



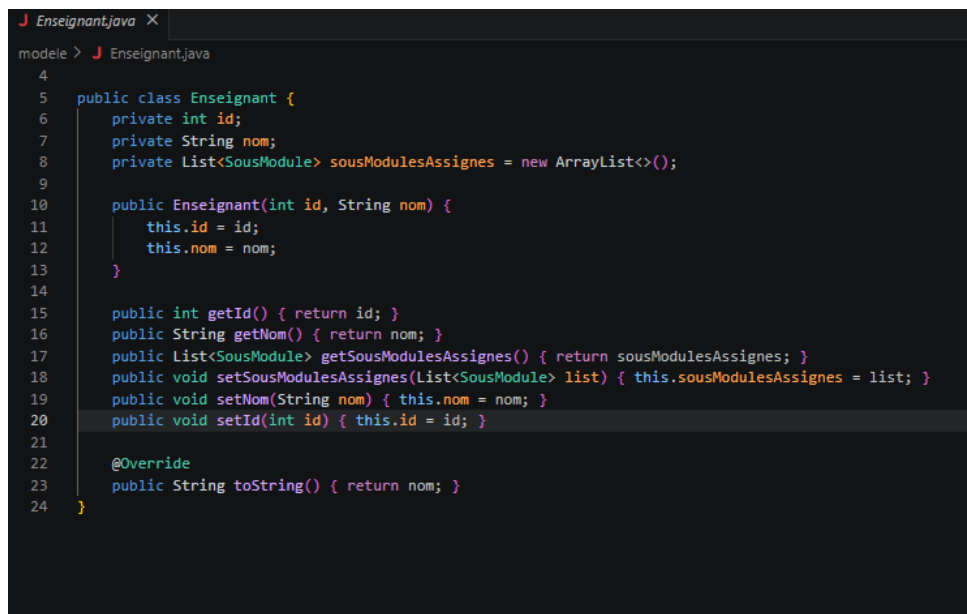
```
J Etudiant.java X
modele > J Etudiant.java > ...
3 public class Etudiant {
35
36     public void setId(int id) { this.id = id; }
37     public void setPrenom(String prenom) { this.prenom = prenom; }
38     public void setNom(String nom) { this.nom = nom; }
39     public void setNumEtudiant(String numEtudiant) { this.numEtudiant = numEtudiant; }
40     public void setArchive(boolean archive) { this.archive = archive; }
41     public void setPromotion(Promotion promotion) { this.promotion = promotion; }
42     public void setDateNaissance(String dateNaissance) { this.dateNaissance = dateNaissance; }
43     public void setTelephone(String telephone) { this.telephone = telephone; }
44     public void setEmail(String email) { this.email = email; }
45
46     public String getNomComple() { return prenom + " " + nom; }
47
48     @Override
49     public String toString() { return prenom + " " + nom; }
50     public boolean aConsommeAnneeReserve() { return aConsommeAnneeReserve; }
51     public void setConsommeAnneeReserve(boolean value) { this.aConsommeAnneeReserve = value; }
52 }
```

FIGURE 3 – Classe Etudiant

3.2.3 Classe Enseignant

```
1 public class Enseignant {
2     private int id;
3     private String nom;
4     private List<SousModule> sousModulesAssignes;
5     // Getters et Setters
6 }
```

Listing 3 – Classe Enseignant



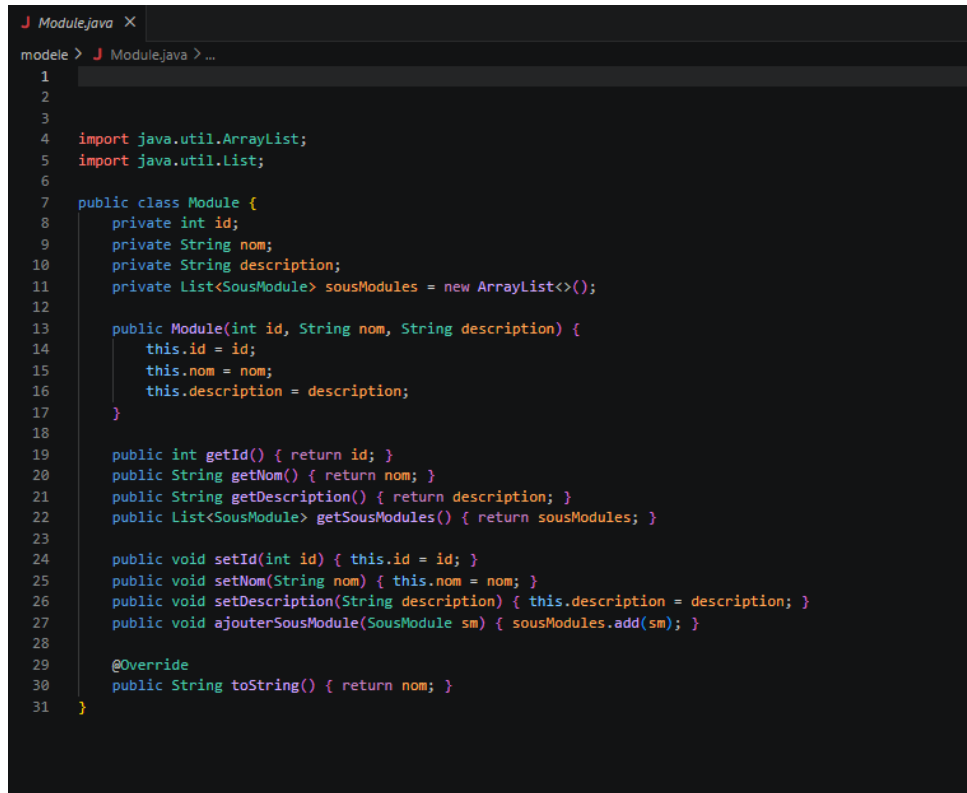
```
J Enseignant.java X
modele > J Enseignant.java
4
5 public class Enseignant {
6     private int id;
7     private String nom;
8     private List<SousModule> sousModulesAssignes = new ArrayList<>();
9
10    public Enseignant(int id, String nom) {
11        this.id = id;
12        this.nom = nom;
13    }
14
15    public int getId() { return id; }
16    public String getNom() { return nom; }
17    public List<SousModule> getSousModulesAssignes() { return sousModulesAssignes; }
18    public void setSousModulesAssignes(List<SousModule> list) { this.sousModulesAssignes = list; }
19    public void setNom(String nom) { this.nom = nom; }
20    public void setId(int id) { this.id = id; }
21
22    @Override
23    public String toString() { return nom; }
24 }
```

FIGURE 4 – Classe Enseignant

3.2.4 Classe Module

```
1 public class Module {  
2     private int id;  
3     private String nom;  
4     private String description;  
5     private List<SousModule> sousModules;  
6     // Getters et Setters  
7 }
```

Listing 4 – Classe Module



The screenshot shows a code editor with the file 'Module.java' open. The code implements the 'Module' class with the following details:

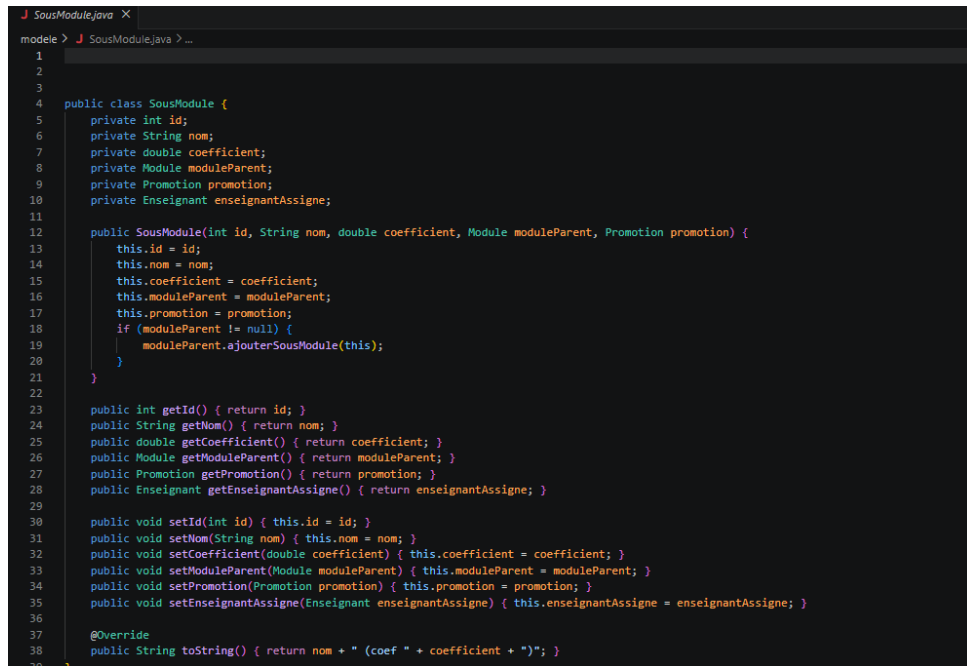
- Imports: `import java.util.ArrayList;` and `import java.util.List;`
- Class Definition: `public class Module {`
- Attributes: `private int id;`, `private String nom;`, `private String description;`, and `private List<SousModule> sousModules = new ArrayList<>();`
- Constructor: `public Module(int id, String nom, String description) {` with assignments `this.id = id;`, `this.nom = nom;`, and `this.description = description;`
- Getters: `public int getId() { return id; }`, `public String getNom() { return nom; }`, `public String getDescription() { return description; }`, and `public List<SousModule> getSousModules() { return sousModules; }`
- Setters: `public void setId(int id) { this.id = id; }`, `public void setNom(String nom) { this.nom = nom; }`, and `public void setDescription(String description) { this.description = description; }`
- Method: `public void ajouterSousModule(SousModule sm) { sousModules.add(sm); }`
- Override: `@Override public String toString() { return nom; }`
- Closing: `}`

FIGURE 5 – Classe Module

3.2.5 Classe SousModule

```
1 public class SousModule {  
2     private int id;  
3     private String nom;  
4     private double coefficient;  
5     private Module moduleParent;  
6     private Promotion promotion;  
7     private Enseignant enseignantAssigne;  
8     // Getters et Setters  
9 }
```

Listing 5 – Classe SousModule

A screenshot of a code editor window titled 'SousModule.java'. The code defines a public class 'SousModule' with several private attributes: 'id' (int), 'nom' (String), 'coefficient' (double), 'moduleParent' (Module), 'promotion' (Promotion), and 'enseignantAssigne' (Enseignant). It includes a constructor that initializes these attributes and calls 'ajouterSousModule' on the parent module if it's not null. There are also getter and setter methods for each attribute, and an overridden 'toString' method that returns the module's name followed by its coefficient in parentheses.

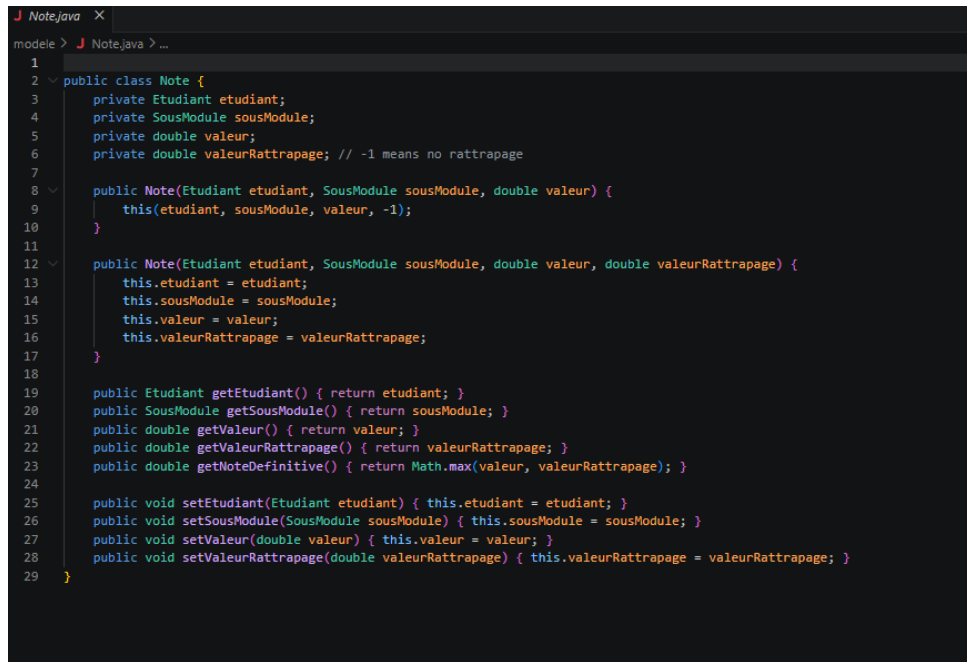
```
1
2
3
4 public class SousModule {
5     private int id;
6     private String nom;
7     private double coefficient;
8     private Module moduleParent;
9     private Promotion promotion;
10    private Enseignant enseignantAssigne;
11
12    public SousModule(int id, String nom, double coefficient, Module moduleParent, Promotion promotion) {
13        this.id = id;
14        this.nom = nom;
15        this.coefficient = coefficient;
16        this.moduleParent = moduleParent;
17        this.promotion = promotion;
18        if (moduleParent != null) {
19            moduleParent.ajouterSousModule(this);
20        }
21    }
22
23    public int getId() { return id; }
24    public String getNom() { return nom; }
25    public double getCoefficient() { return coefficient; }
26    public Module getModuleParent() { return moduleParent; }
27    public Promotion getPromotion() { return promotion; }
28    public Enseignant getEnseignantAssigne() { return enseignantAssigne; }
29
30    public void setId(int id) { this.id = id; }
31    public void setNom(String nom) { this.nom = nom; }
32    public void setCoefficient(double coefficient) { this.coefficient = coefficient; }
33    public void setModuleParent(Module moduleParent) { this.moduleParent = moduleParent; }
34    public void setPromotion(Promotion promotion) { this.promotion = promotion; }
35    public void setEnseignantAssigne(Enseignant enseignantAssigne) { this.enseignantAssigne = enseignantAssigne; }
36
37    @Override
38    public String toString() { return nom + " (coef " + coefficient + ")"; }
39 }
```

FIGURE 6 – Classe SousModule

3.2.6 Classe Note

```
1 public class Note {
2     private Etudiant etudiant;
3     private SousModule sousModule;
4     private double valeur;
5     private double valeurRattrapage;
6     // Getters et Setters
7 }
```

Listing 6 – Classe Note

A screenshot of a Java IDE window titled 'J Note.java'. The code defines a 'Note' class with private attributes 'etudiant', 'sousModule', 'valeur', and 'valeurRattrapage'. It includes two constructors, several getters, and several setters. The code is as follows:

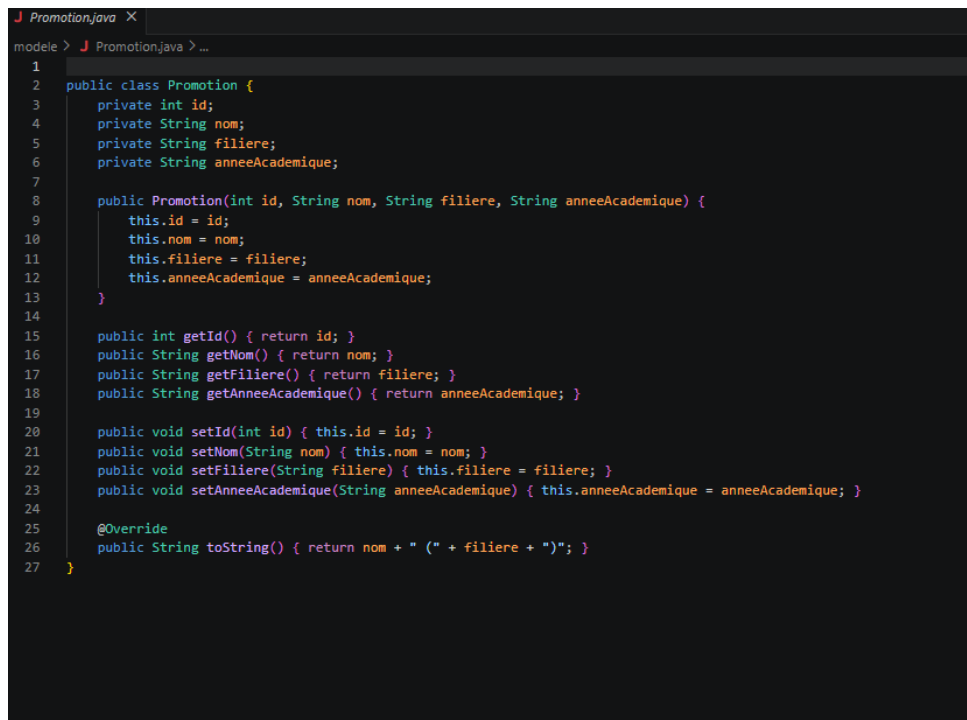
```
1
2 public class Note {
3     private Etudiant etudiant;
4     private SousModule sousModule;
5     private double valeur;
6     private double valeurRattrapage; // -1 means no rattrapage
7
8     public Note(Etudiant etudiant, SousModule sousModule, double valeur) {
9         this(etudiant, sousModule, valeur, -1);
10    }
11
12    public Note(Etudiant etudiant, SousModule sousModule, double valeur, double valeurRattrapage) {
13        this.etudiant = etudiant;
14        this.sousModule = sousModule;
15        this.valeur = valeur;
16        this.valeurRattrapage = valeurRattrapage;
17    }
18
19    public Etudiant getEtudiant() { return etudiant; }
20    public SousModule getSousModule() { return sousModule; }
21    public double getValeur() { return valeur; }
22    public double getValeurRattrapage() { return valeurRattrapage; }
23    public double getNoteDefinitive() { return Math.max(valeur, valeurRattrapage); }
24
25    public void setEtudiant(Etudiant etudiant) { this.etudiant = etudiant; }
26    public void setSousModule(SousModule sousModule) { this.sousModule = sousModule; }
27    public void setValeur(double valeur) { this.valeur = valeur; }
28    public void setValeurRattrapage(double valeurRattrapage) { this.valeurRattrapage = valeurRattrapage; }
29 }
```

FIGURE 7 – Classe Note

3.2.7 Classe Promotion

```
1 public class Promotion {
2     private int id;
3     private String nom;
4     private String filiere;
5     private String anneeAcademique;
6     // Getters et Setters
7 }
```

Listing 7 – Classe Promotion



```
1
2 public class Promotion {
3     private int id;
4     private String nom;
5     private String filiere;
6     private String anneeAcademique;
7
8     public Promotion(int id, String nom, String filiere, String anneeAcademique) {
9         this.id = id;
10        this.nom = nom;
11        this.filiere = filiere;
12        this.anneeAcademique = anneeAcademique;
13    }
14
15    public int getId() { return id; }
16    public String getNom() { return nom; }
17    public String getFiliere() { return filiere; }
18    public String getAnneeAcademique() { return anneeAcademique; }
19
20    public void setId(int id) { this.id = id; }
21    public void setNom(String nom) { this.nom = nom; }
22    public void setFiliere(String filiere) { this.filiere = filiere; }
23    public void setAnneeAcademique(String anneeAcademique) { this.anneeAcademique = anneeAcademique; }
24
25    @Override
26    public String toString() { return nom + " (" + filiere + ")"; }
27 }
```

FIGURE 8 – Classe Promotion

4 Structure de la base de données

4.1 Schéma de la base de données

Voici les principales tables de la base de données `gestion_notes` :

TABLE 2 – Tables de la base de données

Table	Description
utilisateur	Gestion des comptes utilisateurs
promotion	Gestion des promotions
etudiant	Informations des étudiants
enseignant	Informations des enseignants
module	Gestion des modules
sous_module	Gestion des sous-modules
note	Stockage des notes
validation_promotion	Validation des années

4.2 Requêtes principales

4.2.1 Authentification

Listing 8 – Requête d’authentification

```
SELECT * FROM utilisateur  
WHERE nom = ? AND mot_de_passe = ?
```

4.2.2 Liste des étudiants

Listing 9 – Récupération des étudiants

```
SELECT * FROM etudiant  
WHERE promotion_id = ? AND archive = false
```



```

        sommeCoef += sm.getCoefficient();
    }
}
return sommeCoef > 0 ? sommeNotes / sommeCoef : -1;
}

// Calcul moyenne sous-module
public static double calculerMoyenneSousModule(SousModule sm, Etudiant etudiant, StockageMySQL db) {
    Note note = trouverNote(etudiant, sm, db);
    if (note == null) return -1;
    return note.getNoteDefinitive();
}

// Calcul moyenne générale
public static double calculerMoyenneGenerale(List<Module> modules, Etudiant etudiant, StockageMySQL db) {
    double sommeNotes = 0;
    int nbModules = 0;
    for (Module m : modules) {
        double moyModule = calculerMoyenneModule(m, etudiant, db);
        if (moyModule >= 0) {
            sommeNotes += moyModule;
            nbModules++;
        }
    }
    return nbModules > 0 ? sommeNotes / nbModules : -1;
}

// Validation année
public static ResultatValidation validerAnnee(Etudiant etudiant, List<Module> modules, StockageMySQL db) {
    double moyenneGenerale = calculerMoyenneGenerale(modules, etudiant, db);
    int modulesNonValides = 0;
    boolean noteModuleInferieure6 = false;

    for (Module m : modules) {
        if (!estModuleValide(m, etudiant, db)) {
            modulesNonValides++;
        }
        double moyModule = calculerMoyenneModule(m, etudiant, db);
        if (moyModule < 6 && moyModule >= 0) {
            noteModuleInferieure6 = true;
        }
    }

    boolean valide = (moyenneGenerale >= 11) && (modulesNonValides <= 2) && !noteModuleInferieure6;
}

```

FIGURE 10 – Suite du calcul des moyennes

```

    return new ResultatValidation(valide, moyenneGenerale, modulesNonValides, noteModuleInferieure6);
}

// Trouver note
private static Note trouverNote(Etudiant etudiant, SousModule sm, StockageMySQL db) {
    for (Note n : db.getNotesByEtudiant(etudiant.getId())) {
        if (n.getSousModule().getId() == sm.getId()) {
            return n;
        }
    }
    return null;
}

// Classe résultat
public static class ResultatValidation {
    public boolean valide;
    public double moyenneGenerale;
    public int modulesNonValides;
    public boolean noteModuleInferieure6;

    public ResultatValidation(boolean valide, double moyenneGenerale, int modulesNonValides, boolean noteModuleInferieure6) {
        this.valide = valide;
        this.moyenneGenerale = moyenneGenerale;
        this.modulesNonValides = modulesNonValides;
        this.noteModuleInferieure6 = noteModuleInferieure6;
    }
}
}

```

FIGURE 11 – Validation et résultat

5.3 Algorithme de validation

```
1 public static boolean estModuleValide(Module module, Etudiant etudiant,  
   StockageMySQL db) {  
2     double moyenneModule = calculerMoyenneModule(module, etudiant, db);  
3     if (moyenneModule < 11) return false;  
4  
5     for (SousModule sm : getSousModules(module, db)) {  
6         Note note = trouverNote(etudiant, sm, db);  
7         if (note != null && note.getNoteDefinitive() < 6) return false;  
8         if (note == null) return false;  
9     }  
10    return true;  
11 }
```

Listing 10 – Validation d'un module

6 Interface utilisateur

6.1 Page de connexion

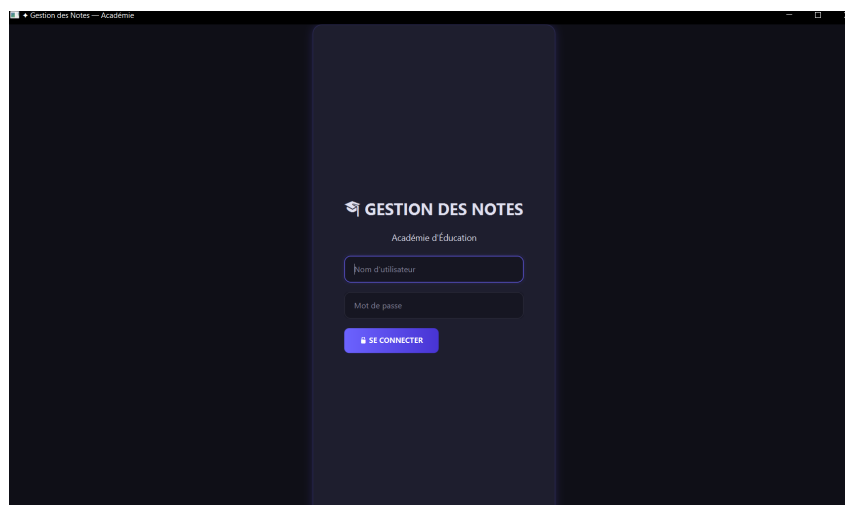


FIGURE 12 – Interface de connexion

6.2 Interface principale

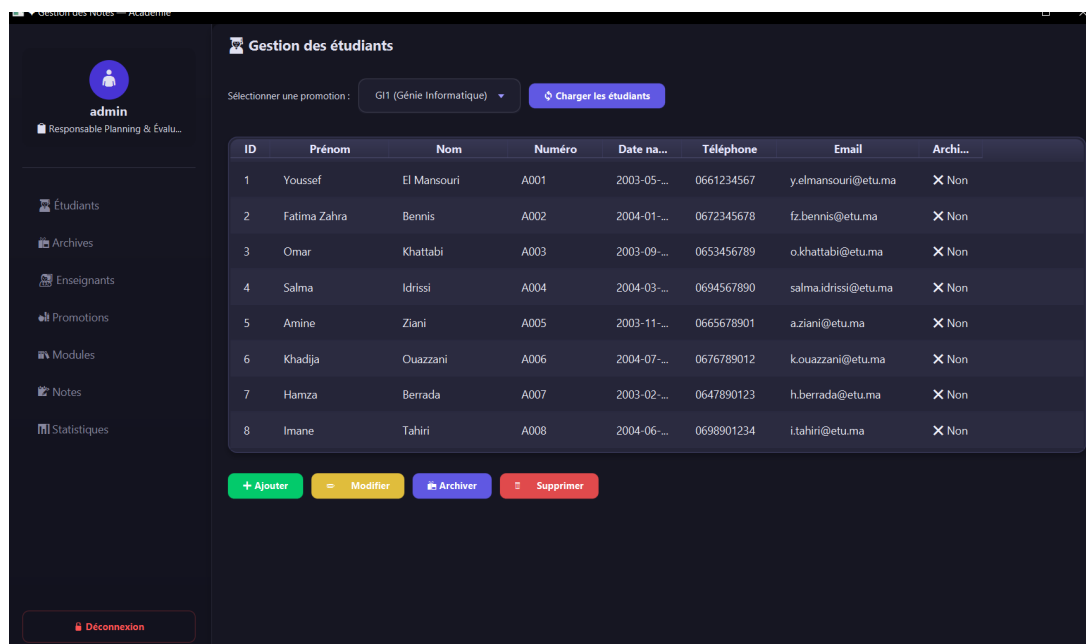


FIGURE 13 – Interface principale - Gestion des étudiants

6.3 Gestion des étudiants

L'interface permet de :

- Ajouter, modifier, archiver et supprimer des étudiants
- Filtrer par promotion
- Visualiser les informations complètes des étudiants

6.4 Statistiques

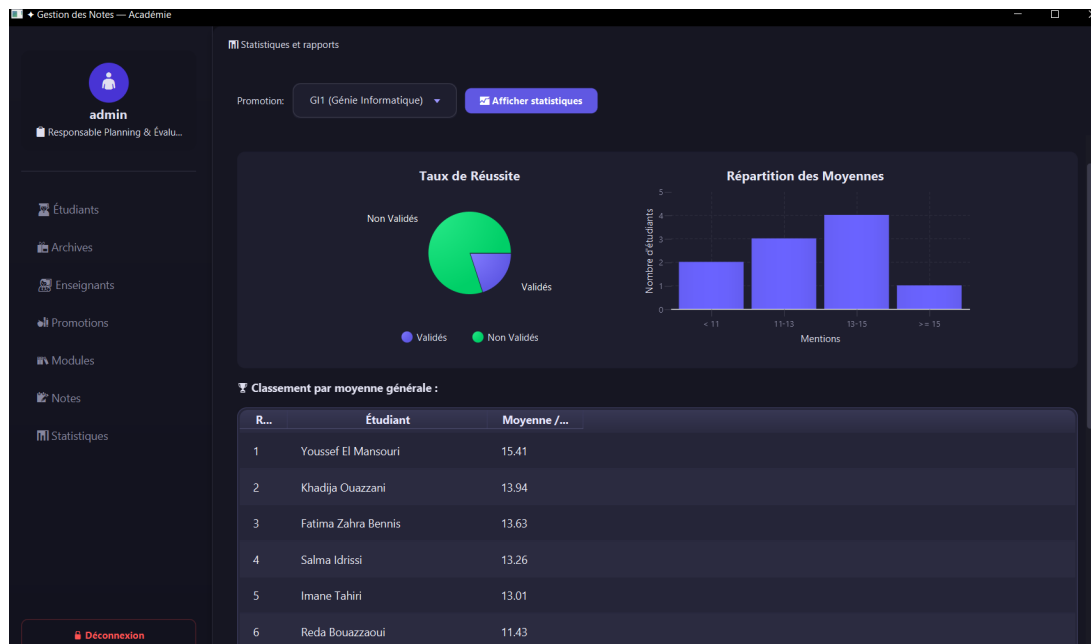


FIGURE 14 – Statistiques et rapports

- **Meilleur étudiant** : Affichage du meilleur étudiant de la promotion
- **Classement** : Classement par moyenne générale
- **Taux de réussite** : Graphique circulaire des validés/non validés
- **Répartition des moyennes** : Histogramme des tranches de moyennes

6.5 Structure du menu

TABLE 3 – Fonctionnalités par rôle

Rôle	Fonctionnalités
PLANNING	Étudiants, Enseignants, Archives, Promotions, Modules, Notes, Statistiques
RESPONSABLE_FILIERE	Étudiants, Archives, Validation Année, Statistiques
ENSEIGNANT	Saisie des notes
ETUDIANT	Mes notes, Ma progression

7 Import/Export Excel

7.1 Fonctionnalités

L'application intègre la bibliothèque Apache POI pour l'import/export des notes :

- **Export** : Génération de fichiers Excel avec les notes
- **Import** : Chargement des notes depuis un fichier Excel
- **Format** : Prise en charge des fichiers .xlsx et .xls

7.2 Structure du fichier Excel

TABLE 4 – Format du fichier Excel d'import/export

Colonne	Contenu
Colonne 1	Nom de l'étudiant
Colonne 2	Note initiale /20
Colonne 3	Rattrapage /20

7.3 Code d'export

```
1 private void exporterNotesExcel(ObservableList<NoteTableRow> data,  
   SousModule sm) {  
2     FileChooser fileChooser = new FileChooser();  
3     fileChooser.setTitle("Exporter les notes vers Excel");  
4     fileChooser.getExtensionFilters().add(  
5         new FileChooser.ExtensionFilter("Excel Files", "*.xlsx")  
6     );  
7     // ...  
8 }
```

Listing 11 – Export vers Excel

8 Détails de la base de données

8.1 Table Utilisateur

id	nom	mot_de_passe	role	filiere
1	admin	admin123	PLANNING	NULL
2	Dr. Bennani	filiere123	RESPONSABLE_FILIERE	Génie Informatique
4	Prof. Alaoui	prof123	ENSEIGNANT	NULL
9	ElMansouriYoussef	...	ETUDIANT	NULL

FIGURE 15 – Extrait de la table utilisateur

8.2 Table Étudiant

id	prenom	nom	num_etudiant	promotion_id	archive
1	Youssef	El Mansouri	A001	1	0
2	Fatima Zahra	Bennis	A002	1	0
3	Omar	Khattabi	A003	1	0

FIGURE 16 – Extrait de la table etudiant

9 Conclusion

9.1 Bilan du projet

Le projet de gestion des notes a permis de développer une application complète répondant aux besoins des établissements d'enseignement supérieur. Les fonctionnalités implémentées couvrent l'ensemble du cycle de gestion des évaluations.

9.2 Points forts

- Interface utilisateur moderne et intuitive
- Gestion multi-rôles avec des permissions adaptées
- Système de validation conforme au règlement d'évaluation
- Import/Export des données via Excel
- Statistiques et rapports détaillés

10 Annexes

10.1 Configuration du projet

```
1 {  
2     "java.project.sourcePaths": ["modele"],  
3     "java.project.outputPath": "bin",  
4     "java.project.referencedLibraries": [  
5         "lib/mysql-connector-j-9.7.0.jar",  
6         "lib/poi-5.2.5.jar",  
7         "lib/poi-ooxml-5.2.5.jar",  
8         "C:\\javafx-sdk-26.0.1\\lib\\*"  
9     ],  
10    "java.debug.settings.vmArgs": "--module-path \"C:\\javafx-sdk  
11    -26.0.1\\lib\" --add-modules javafx.controls,javafx.fxml"  
12 }
```

Listing 12 – settings.json

10.2 Dépendances principales

- MySQL Connector/J 9.7.0
- Apache POI 5.2.5 (Import/Export Excel)
- JavaFX 26.0.1